

HW04 — Аналіз і порівняння методів тестування безпеки

1. Лабораторна частина: DAST з OWASP ZAP

DAST-перевірка виконувалася за допомогою OWASP ZAP Desktop. Сканування було локальним: Juice Shop працював у Docker/WSL, а OWASP ZAP Desktop — у Windows, тому для DAST використовувалась WSL-адреса сервісу `http://172.27.224.80:3000/`.

Матеріали для перевірки результатів:

- HTML-звіт ZAP: evidence/tool-runs/zap_results.html;
- зведення виявлень ZAP для цільового сервісу: <evidence/tool-runs/zap-alerts-summary.md>;
- скриншоти запуску й результатів: [01-juice-shop-running.png](#), [02-zap-scan-running.png](#), [03-zap-alert-summary.png](#), [04-zap-selected-finding.png](#).

Для аналізу взято виявлення, що стосуються локального екземпляра Juice Shop за адресою `http://172.27.224.80:3000/`.

№	Виявлення OWASP ZAP	Ризик / достовірність	Приклад URL
1	SQL Injection	High / Low	GET /rest/products/search?q=%27%28
2	Content Security Policy Header Not Set	Medium / High	GET /
3	Cross-Domain Misconfiguration	Medium / Medium	GET /chunk-24EZLZ4I.js
4	Session ID in URL Rewrite	Medium / High	GET /socket.io/?...&sid=oCSgl27LzFT4KybHAAAC
5	Missing Anti-clickjacking Header	Medium / Medium	POST /socket.io/?...&sid=oCSgl27LzFT4KybHAAAC
6	X-Content-Type-Options Header Missing	Low / Medium	GET /socket.io/?EI0=4&transport=polling...

Коментарі до DAST-виявлень

1. **SQL Injection.** ZAP отримав 500 Internal Server Error на ін'єкційний пошуковий запит. Це показує, що DAST добре знаходить проблеми, які проявляються через реальну HTTP-поведінку застосунку.
2. **Content Security Policy Header Not Set.** Відсутній CSP-заголовок. Для SPA це важливо, бо CSP зменшує наслідки XSS та помилок у роботі з HTML/JavaScript.
3. **Cross-Domain Misconfiguration.** ZAP виявив потенційно небезпечну міждоменну поведінку (cross-origin) на ресурсах застосунку. Це варто перевіряти

в передрелізному середовищі (staging), бо CORS-помилки часто залежать від конфігурації під час виконання (runtime).

4. **Session ID in URL Rewrite.** Ідентифікатор сесії потрапляє в URL. Такі значення можуть залишатися в логах, історії браузера або referrer-заголовках.
5. **Missing Anti-clickjacking Header.** Відсутність X-Frame-Options або відповідної CSP-директиви може дозволити відкривати сторінку у frame та створювати clickjacking-сценарії.
6. **X-Content-Type-Options Header Missing.** Відсутній X-Content-Type-Options: nosniff. Це не найкритичніша проблема, але вона показує нестачу базового посилення захисту (hardening) для HTTP-відповідей.

Головна користь DAST у цій лабораторній роботі — перевірка застосунку як “чорної скриньки”: ZAP бачить реальні відповіді, заголовки, URL, статуси та поведінку API/SPA після запуску. Це особливо корисно для пошуку помилок конфігурації, проблем із заголовками безпеки (security headers), поведінки під час виконання та підтвердження того, що вразливість справді проявляється через HTTP.

2. Лабораторна частина: SAST з Semgrep

SAST-перевірка виконувалася Semgrep по локальному коду Juice Shop. Semgrep запускався в Docker-контейнері semgrep/semgrep:latest.

Матеріали для перевірки результатів:

- JSON-звіт Semgrep: [reports/results_semgrep.json](#);
- TSV-вибірка результатів: [evidence/tool-runs/semgrep-findings.tsv](#);
- скриншот завершення команди: [05-semgrep-command-finished.png](#).

Semgrep знайшов 23 результати. Нижче подані чотири різні за типом приклади, які добре показують сильні сторони SAST.

№	Результат Semgrep	Файл / місце	Чому це важливо
1	SQL-ін'єкція (SQL injection) у Sequelize	src/routes/search.ts: 23, також src/routes/login.ts:34	Дані з HTTP-запиту потрапляють у SQL/ORM-запит. На відміну від DAST, SAST показує місце в коді, де треба перейти на параметризовані запити або безпечний query builder.
2	Жорстко прописаний JWT-секрет (hardcoded JWT secret)	src/lib/insecurity.ts:56	Секрет для JWT зберігається в коді. У реальному SSDLC такі значення мають бути в сховищі секретів або змінних середовища, а не в репозиторії.

№	Результат Semgrep	Файл / місце	Чому це важливо
3	Обхід шляхів (path traversal) через <code>res.sendFile</code>	<code>src/routes/</code> <code>fileServer.ts:33</code> , також <code>src/routes/</code> <code>keyServer.ts:14</code> , <code>src/</code> <code>routes/</code> <code>logfileServer.ts:14</code> , <code>src/routes/</code> <code>quarantineServer.ts:</code> <code>14</code>	Користувачський ввід впливає на шлях до файлу. Це може дозволити читання неочікуваних файлів, якщо шлях не нормалізується і не обмежується дозволеною директорією.
4	Необроблений HTML (raw HTML) / можливий XSS	<code>src/routes/</code> <code>chatbot.ts:205</code>	Дані потрапляють у HTML, який формується вручну. Для такого коду потрібне екранування, санітизація або безпечна генерація HTML.

SAST корисний до запуску застосунку: його можна виконувати на PR, у CI або локально перед злиттям змін. Він показує конкретні рядки коду, тому добре підходить для швидкого виправлення дефектів на ранніх етапах SSDLC.

3. Порівняння SAST і DAST

Критерій	SAST	DAST
Що аналізує	Код, конфігурацію, workflow-файли	Запущений застосунок через HTTP/API
Коли запускати	На етапі розробки, commit, PR, CI	У тестовому або передрелізному середовищі після деплою
Сильна сторона	Показує точний файл і рядок проблеми	Показує реальну поведінку під час виконання та вплив конфігурації
Обмеження	Може не знати, чи код реально досяжний під час виконання	Не завжди показує конкретне місце в коді
Приклад з цієї роботи	Semgrep знайшов SQL-ін'єкцію у <code>src/routes/search.ts</code> і жорстко прописаний JWT-секрет	ZAP знайшов SQL Injection через <code>/rest/products/search</code> і відсутні заголовки безпеки

У цій роботі SAST і DAST доповнюють один одного. Наприклад, Semgrep показує місця в коді, де можливі SQL-ін'єкції, а ZAP підтверджує, що одна з пошукових кінцевих точок API реагує на ін'єкційний рядок (payload) помилкою сервера. Так само Semgrep добре знаходить жорстко прописані секрети та обхід шляхів у коді, а ZAP краще бачить HTTP-заголовки, session ID у URL та CSP/CORS-конфігурацію.

4. Таблиця вразливостей Juice Shop і SSDLC

№	Вразливість / ризик	Метод	Інструмент	Етап SSDLC
1	SQL Injection у пошуку товарів	SAST + DAST	Semgrep, OWASP ZAP	Розробка, CI, передрелізне середовище
2	Жорстко прописаний JWT-секрет	SAST	Semgrep	Commit, CI, релізний контрольний гейт
3	Обхід шляхів у файлових маршрутах	SAST	Semgrep	Розробка, перевірка коду, CI
4	Необроблений HTML / можливий XSS у chatbot route	SAST, додатково DAST/IAST	Semgrep, OWASP ZAP	Розробка, інтеграційне тестування
5	Відсутні CSP / anti-clickjacking / посилення Content-Type	DAST	OWASP ZAP	Передрелізне середовище, перевірка перед релізом
6	Session ID in URL Rewrite	DAST	OWASP ZAP	Інтеграційне тестування, передрелізне середовище

Рекомендовані дії для SSDLC

- SQL Injection у пошуку товарів.** Використати параметризовані запити, додати unit/integration тести на ін'єкційні рядки, запускати SAST у PR і DAST перед релізом.
- Жорстко прописаний JWT-секрет.** Прибрати секрет із коду, зберігати його в сховищі секретів або змінних середовища, додати перевірку секретів у CI.
- Обхід шляхів у файлових маршрутах.** Канонізувати шлях, перевіряти allowlist директорій, заборонити . . та небезпечні сегменти, додати негативні тести.
- Необроблений HTML / можливий XSS у chatbot route.** Екранувати користувацькі дані, використовувати безпечні шаблони або санітизацію HTML.
- Відсутні CSP / anti-clickjacking / посилення Content-Type.** Додати CSP, X-Frame-Options або frame-ancestors, X-Content-Type-Options: nosniff, перевіряти заголовки після деплою.
- Session ID in URL Rewrite.** Не передавати session ID через URL, використовувати cookie з HttpOnly, Secure, SameSite, зменшити витік через логи та referrer.

Найкращий результат дає не один інструмент, а комбінація контрольних гейтів: SAST рано знаходить дефекти в коді, DAST перевіряє реальний деплой, а ручна перевірка допомагає оцінити бізнес-контекст і пріоритет виправлення.

5. IAST або RASP: практичний сценарій

Для Juice Shop практичним доповненням був би IAST-сценарій у тестовому середовищі. IAST підключається до запущеного застосунку й аналізує, як дані реально проходять через код під час інтеграційних або end-to-end тестів.

Приклад сценарію:

1. Juice Shop запускається у тестовому або передрелізному середовищі.
2. До Node.js/Express застосунку підключається IAST agent.
3. Автоматизовані тести або тестувальник проходять основні сценарії: пошук товарів, login, chatbot, завантаження файлів, redirect, websocket/polling.
4. IAST фіксує потік даних (data flow) від HTTP-запиту до небезпечного приймача (sink): SQL query, HTML response, file path, redirect або token/session handling.
5. Результат потрапляє до черги завдань (backlog) з прив'язкою до кінцевої точки API (endpoint), стека викликів і критичності.

IAST був би корисним саме між SAST і DAST. Він краще за SAST розуміє, чи код реально виконується, і краще за DAST показує внутрішній шлях даних. Для навчального Juice Shop це допомогло б точніше підтвердити SQL-ін'єкції, XSS-подібні ризики та обхід шляхів.

RASP більше підходить для захисту критичних систем у production. Наприклад, RASP може блокувати підозрілі SQL-рядки атаки або обхід шляхів під час виконання. Але для цієї лабораторної роботи IAST доречніший, бо завдання фокусується на аналізі та порівнянні методів тестування.

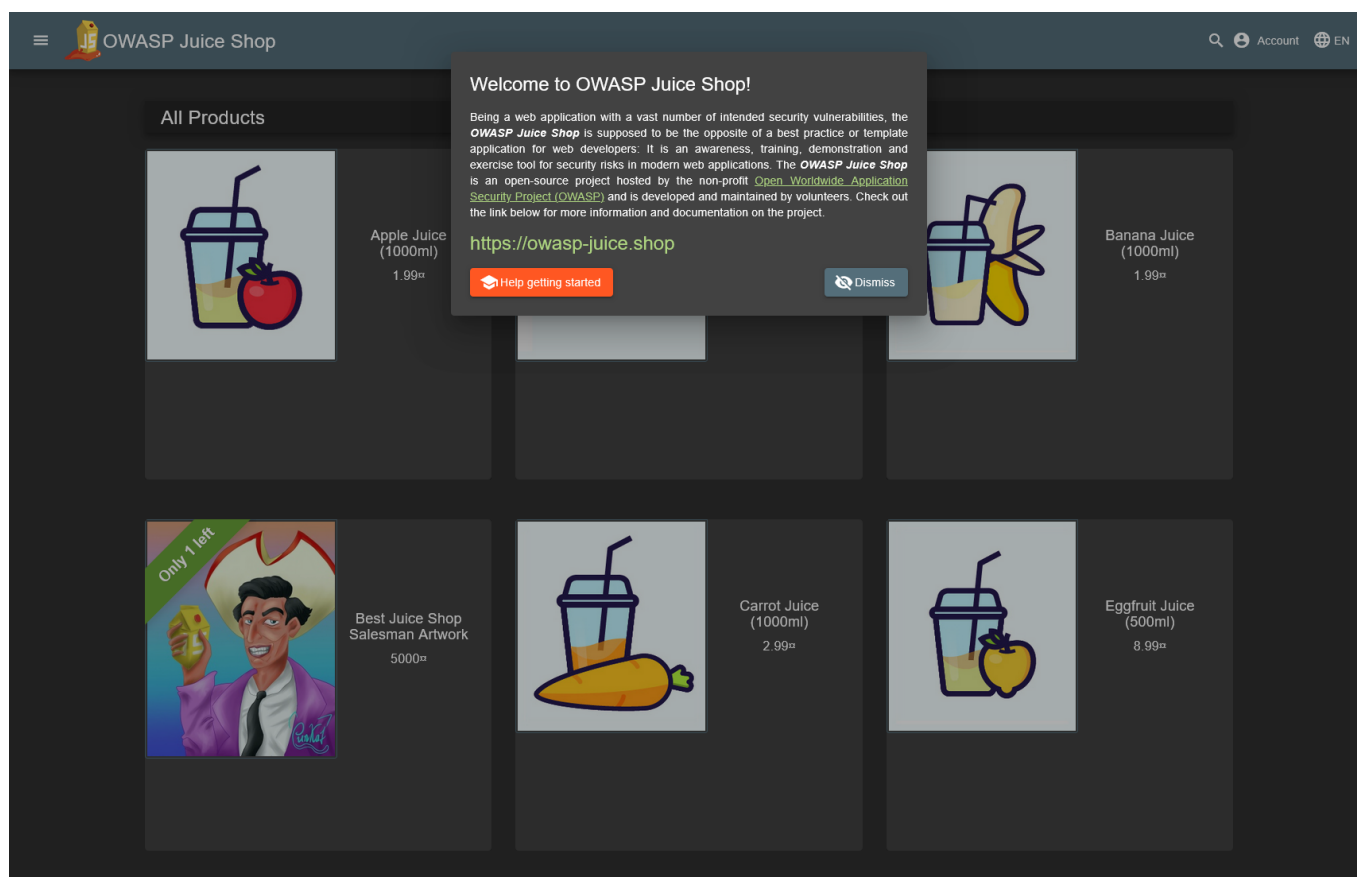
6. Вибір методів тестування для різних типів застосунків

Тип застосунку	Рекомендовані методи	Як інтегрувати
REST API	SAST, DAST, за можливості IAST	SAST запускати на PR і в CI; DAST запускати в передрелізному середовищі за OpenAPI-специфікацією або набором API-запитів; IAST використовувати разом з integration tests для критичних кінцевих точок.

Тип застосунку	Рекомендовані методи	Як інтегрувати
SPA / вебзастосунок	SAST, DAST, ручна перевірка заголовків безпеки	SAST перевіряє TypeScript/JavaScript, серверні маршрути і CI workflow; DAST перевіряє заголовки у відповідях запущеного застосунку, CORS/CSP, XSS-поведінку, роботу з сесіями; перед релізом корисно мати окремий контрольний список безпеки для браузерних ризиків.

Для REST API головний акцент — валідація вхідних даних, авторизація, SQL/NoSQL injection, помилки доступу до ресурсів і безпечна робота з секретами. Для SPA важливі XSS, CSP, CORS, clickjacking, ризики залежностей і коректна робота з токенами. В обох випадках SAST краще ставити ближче до розробника, а DAST — ближче до реального середовища.

Додаток: скриншоти виконання



Скриншот 1. Локальний OWASP Juice Shop запущений у Docker/WSL.

Unbenannte Session - ZAP 2.17.0

Standard Mode

Sites

Automated Scan

This screen allows you to launch an automated scan against an application - just enter its URL below and press 'Attack'.

Bitte beachten Sie, dass Sie nur Anwendungen angreifen, für die Sie eine besondere Erlaubnis zum Testen erhalten haben.

URL to attack: Auswählen...

Use traditional spider: ☒

Use ajax spider: ☐ If Modern with

Fortschritt: Actively scanning (attacking) the URLs discovered by the spider(s)

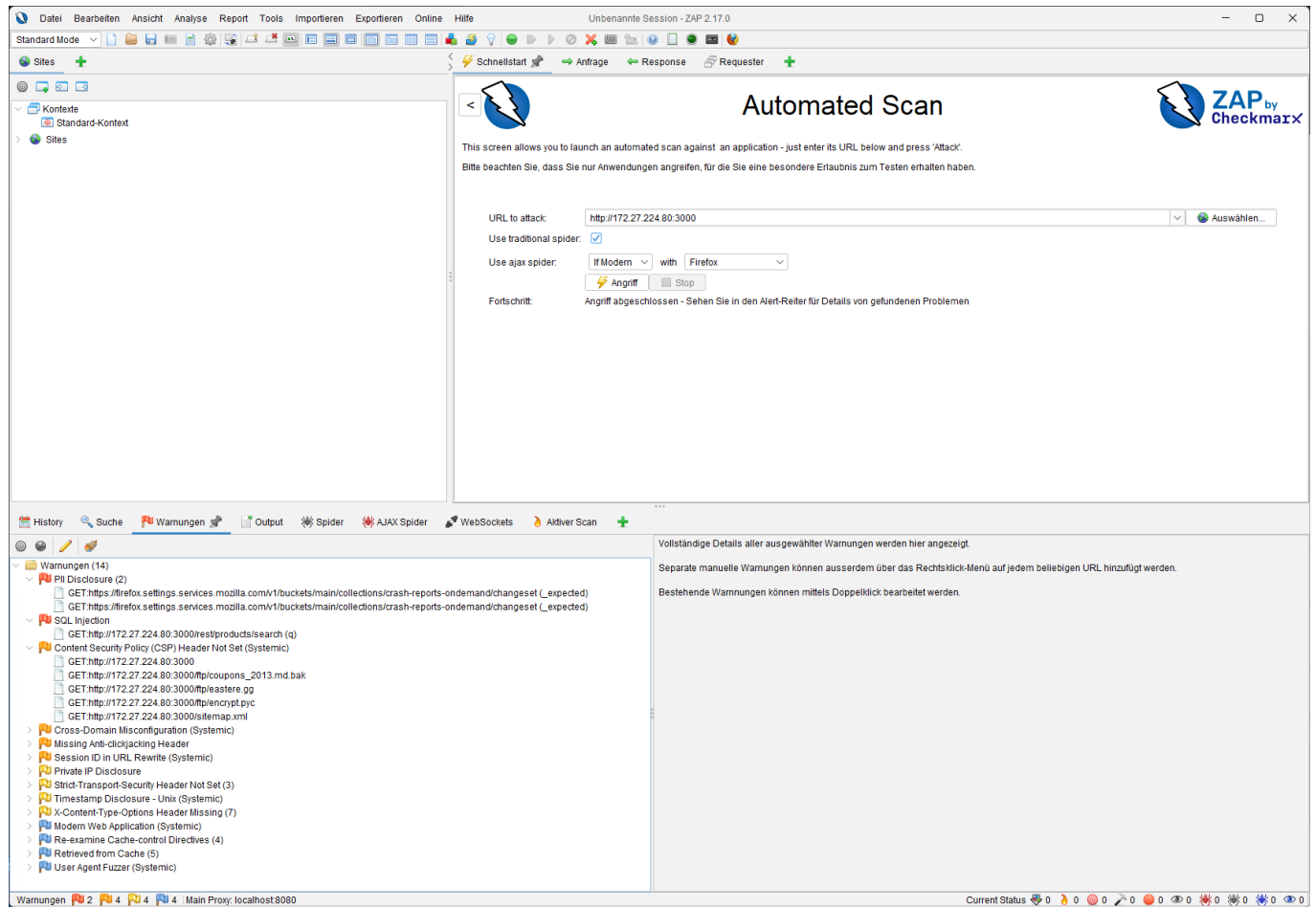
History Suche Warnungen Output Spider AJAX Spider WebSockets Aktiver Scan

New Scan Fortschritt: 0 http://172.27.224.80:3000 35% Laufende Scans: 1 Num Requests: 285 New Alerts: 1 Exportieren

ID	Req. Timestamp	Resp. Timestamp	Methode	URL	Code	Grund	RTT	Size Resp. Header	Size Resp. Body
3.632	05.05.26, 17:00:40	05.05.26, 17:00:41	GET	https://archive.mozilla.org/pub/system-addons/newtabline...	200	OK	473 ms	862 Byte	1.908.141 Byte
3.633	05.05.26, 17:00:47	05.05.26, 17:00:48	GET	https://archive.mozilla.org/pub/system-addons/newtabline...	200	OK	283 ms	861 Byte	1.908.141 Byte
3.634	05.05.26, 17:00:47	05.05.26, 17:00:48	GET	https://archive.mozilla.org/pub/system-addons/newtabline...	200	OK	397 ms	862 Byte	1.908.141 Byte
3.635	05.05.26, 17:00:49	05.05.26, 17:00:50	GET	https://archive.mozilla.org/pub/system-addons/newtabline...	200	OK	237 ms	862 Byte	1.908.141 Byte
3.636	05.05.26, 17:00:53	05.05.26, 17:00:53	GET	https://archive.mozilla.org/pub/system-addons/newtabline...	200	OK	273 ms	862 Byte	1.908.141 Byte
3.637	05.05.26, 17:00:53	05.05.26, 17:00:53	GET	https://archive.mozilla.org/pub/system-addons/newtabline...	200	OK	332 ms	861 Byte	1.908.141 Byte
3.638	05.05.26, 17:00:53	05.05.26, 17:00:53	GET	https://archive.mozilla.org/pub/system-addons/newtabline...	200	OK	369 ms	862 Byte	1.908.141 Byte
3.639	05.05.26, 17:00:56	05.05.26, 17:00:56	GET	https://archive.mozilla.org/pub/system-addons/newtabline...	200	OK	320 ms	862 Byte	1.908.141 Byte
3.640	05.05.26, 17:01:01	05.05.26, 17:01:01	GET	https://archive.mozilla.org/pub/system-addons/newtabline...	200	OK	396 ms	862 Byte	1.908.141 Byte
3.641	05.05.26, 17:01:12	05.05.26, 17:01:12	GET	https://archive.mozilla.org/pub/system-addons/newtabline...	200	OK	307 ms	862 Byte	1.908.141 Byte
3.642	05.05.26, 17:01:12	05.05.26, 17:01:13	GET	https://archive.mozilla.org/pub/system-addons/newtabline...	200	OK	365 ms	861 Byte	1.908.141 Byte
3.643	05.05.26, 17:01:12	05.05.26, 17:01:13	GET	https://archive.mozilla.org/pub/system-addons/newtabline...	200	OK	480 ms	862 Byte	1.908.141 Byte
3.644	05.05.26, 17:01:17	05.05.26, 17:01:17	GET	https://archive.mozilla.org/pub/system-addons/newtabline...	200	OK	319 ms	862 Byte	1.908.141 Byte
3.645	05.05.26, 17:01:19	05.05.26, 17:01:19	GET	https://archive.mozilla.org/pub/system-addons/newtabline...	200	OK	410 ms	862 Byte	1.908.141 Byte
3.646	05.05.26, 17:01:25	05.05.26, 17:01:25	GET	https://archive.mozilla.org/pub/system-addons/newtabline...	200	OK	337 ms	862 Byte	1.908.141 Byte
3.647	05.05.26, 17:01:25	05.05.26, 17:01:26	GET	https://archive.mozilla.org/pub/system-addons/newtabline...	200	OK	300 ms	861 Byte	1.908.141 Byte
3.648	05.05.26, 17:01:26	05.05.26, 17:01:27	GET	https://archive.mozilla.org/pub/system-addons/newtabline...	200	OK	324 ms	861 Byte	1.908.141 Byte
3.649	05.05.26, 17:01:41	05.05.26, 17:01:42	GET	https://archive.mozilla.org/pub/system-addons/newtabline...	200	OK	270 ms	861 Byte	1.908.141 Byte
3.650	05.05.26, 17:01:51	05.05.26, 17:01:51	GET	https://archive.mozilla.org/pub/system-addons/newtabline...	200	OK	292 ms	861 Byte	1.908.141 Byte
3.651	05.05.26, 17:01:53	05.05.26, 17:01:55	GET	https://archive.mozilla.org/pub/system-addons/newtabline...	200	OK	1.67 s	861 Byte	1.908.141 Byte
3.652	05.05.26, 17:02:06	05.05.26, 17:02:06	GET	https://archive.mozilla.org/pub/system-addons/newtabline...	200	OK	279 ms	861 Byte	1.908.141 Byte

Warnungen 2 4 3 Main Proxy: localhost:8080 Current Status 0 1 0 0 0 0 0 0 0 0

Скриншот 2. OWASP ZAP Desktop виконує DAST-перевірку локального сервісу.



Скриншот 3. Панель виявлень OWASP ZAP після сканування.


```
goit@N-BD013: ~/projects/sec x goit@N-BD013: ~/projects/sec x goit@N-BD013: ~/projects/se x + v
6a0ac1617861: Pull complete
250c14aaa567: Pull complete
bd9ddc54bea9: Pull complete
f3a85c50fdbb: Pull complete
5a216db4ba33: Pull complete
559519e2807f: Pull complete
fc80c01c297e: Pull complete
ef124437829b: Pull complete
5b03cc6703a82: Pull complete
b25ddf811014: Pull complete
72a0cc822638: Pull complete
4349ae20a4df: Pull complete
0d9d16429388: Pull complete
5337f13d6dbf: Pull complete
d2d9531a0926: Pull complete
050dd0545bfc: Pull complete
c88b1e003836: Pull complete
535ae49d1ee7: Pull complete
6db4f1233236: Pull complete
b830bf843e58: Pull complete
858c22f111c0: Pull complete
Digest: sha256:7818f1d7884974ab6dda7bef8f4a2c8e165ea2142fd8260515d380e4f1407263
Status: Downloaded newer image for semgrep/semgrep:latest

Scan Status
Scanning 1048 files tracked by git with 544 Code rules:

Language  Rules  Files  Origin  Rules
-----
<multilang> 14    1048  Community  544
ts          71     474
json        3     105
yaml        19     91
html         1     77
solidity     1     17
js           65    13
dockerfile   4      1
bash         1      1

Scan Summary
✔ Scan completed successfully.
• Findings: 23 (23 blocking)
• Rules run: 114
• Targets scanned: 1048
• Parsed lines: ~99.9%
• Scan skipped:
  • Files larger than files 1.0 MB: 8
  • Files matching .semgrepignore patterns: 141
• Scan was limited to files tracked by git
• For a detailed list of skipped files and lines, run semgrep with the --verbose flag
Ran 114 rules on 1048 files: 23 findings.
goit@N-BD013: ~/projects/secure-arch-lab-hw04$
```

Скриншот 5. Завершення запуску Semgrep і збереження результатів.

7. Висновок

У цьому завданні SAST і DAST показали різні, але взаємодоповнювальні сторони безпеки Juice Shop. Semgrep знайшов проблеми в коді: SQL-ін'єкцію, жорстко прописаний JWT-секрет, обхід шляхів і необроблений HTML. OWASP ZAP Desktop показав ризики під час виконання: SQL Injection через HTTP-запит, відсутні заголовки безпеки, session ID в URL та інші проблеми конфігурації.

Для стабільного SSDLC варто запускати SAST рано — на PR/CI, DAST — після деплою в тестове або передрелізне середовище, а IAST додавати для критичних потоків, де потрібно бачити реальний шлях даних у застосунку. Такий підхід зменшує шанс, що вразливість залишиться непоміченою до релізу.